

How the workforce runs

Every agent, every tool, every workflow, and the exact sequence each one follows when the whole system is switched on.

PART 01

What an agent is
The one-job rule
The 10 instruction layers
What it is actually told
The org: tiers and departments

PART 02 · 03

The catalogue of 69 tools
The gate every call passes
The autonomy ladder
One task, turn by turn
Checkpoints and end states

PART 04 · 05 · 06

The clock, minute by minute
Every process, end to end
A day with everything on
Appendix A — the roster
Appendix B — the catalogue

What this document is

Dakyworld OS is not a chatbot with a job list. It is a workforce: 51 agents, each with one craft, each with an employee record, a written instruction, a granted toolkit, and a level of autonomy a person set deliberately. This describes the machine that runs them.

51 AGENTS One deliverable each. Nobody does two jobs.	69 TOOLS 12 reach outside the company; 16 spend money.	10 PROMPT LAYERS Per agent, editable, and read fresh on every task.	60s THE TICK Nine independent jobs, settled separately.
--	---	--	--

There are three documents about this system and they answer three different questions. **The Agent Master Workflow** answers *how does a stranger become a paying client* — the business flow, in nine stages. **The Dakyworld OS Reference** answers *what exactly is each agent told* — every roster row and every instruction printed in full. **This one** answers the question underneath both: *what actually happens*. What runs, in what order, gated by what, recorded where, and stopping for whom.

Everything numbered in it is read out of the code it describes at build time — the roster from `services/agentRegistry.ts`, the catalogue from `services/tools/catalogue.ts`, the writing jobs from `services/writers/registry.ts`, the model routing from `lib/models/registry.ts`. A reference table that has drifted from the system is worse than no table at all: somebody grants a tool that no longer exists, or looks for one added last month and concludes it cannot be done.

The one sentence that explains the whole design

An agent proposes; the gate decides; a person approves what leaves the building. Every agent in the system seeds at autonomy level 1 with dry run switched on, which means it can research, read, reason, draft and prepare — and nothing it decides takes effect until somebody raises that level or approves that specific action. Everything in Part O2 exists to make that promise true in code rather than in a prompt.

HOW TO READ THE REST

Parts O1 to O3 are the machine: what an agent is, what it may call, and what happens between a task being raised and a person reading the result. Part O4 is the clock — the part that happens whether or not anybody opens the app, and therefore the part worth being certain about before an autonomy level is raised. Part O5 walks every process the system runs, end to end. Part O6 puts them together into one ordinary day with everything switched on.

PART 01

What an agent is

An employee record, not a naked prompt. A job with one deliverable, a written instruction in ten layers, a toolkit somebody granted, a level of autonomy somebody set, and a memory of what it concluded last time.

IN THIS PART

The employee record · The one-job rule · The ten instruction layers · The eight regions of an assembled prompt · Memory and context, and why they are not the same thing · The org chart in numbers

The employee record

Every agent is a row in Agent, seeded from `services/agentRegistry.ts` and never overwritten by a deploy.

KEY	The stable identifier the seeds, the event router and the mail room address it by. <code>outreach.writer</code> , <code>billing.collector</code> , <code>review.look</code> .
NAME & TITLE	What a person calls it. The name on a task card and on the audit PDF.
TIER	Board, Executive, Governance, Operational or Specialist. Decides what kind of thing it produces — a decision, or a deliverable.
DEPARTMENT	Which part of the business it belongs to. Nine of them.
MANAGERKEY	Self-reference by key. This is the delegation graph: an agent may only hand work down to a direct report . A <code>managerKey</code> pointing at nobody silently breaks <code>delegate</code> , which is why a roster check exists for it.
MISSION	The one thing it exists to produce, in a sentence.
SKILLS	What it is good at, in a client's words rather than a tool key's. Read by <code>findAgent</code> , so “who do I ask for a video edit” has an answer.
TOOLKIT	The grant. Tool keys this agent may call. An agent with an empty toolkit can call nothing at all.
AUTONOMYLEVEL	0 observe · 1 draft · 2 recommend · 3 execute-with-policy · 4 autonomous · 5 delegated. Seeds at 1.
DRYRUN	Sits above the level: on means nothing it decides takes effect. Seeds on .
PROMPT	The 10 layered sections. Editable, and <code>promptEditedAt</code> marks a rewritten seed so a deploy cannot undo it.
STATUS	DRAFT, ACTIVE, PAUSED or RETIRED. Pausing an agent is how standing work is stopped; a paused or retired agent is refused at the gate and is not a routing destination.

A deploy never overwrites an edit

`ensureAgents()` only ever *creates*. New agents arrive on deploy; re-wording an existing one in the seed file changes nothing on a live database. That is deliberate — a deploy must not undo an edit the founder made to words that go out under his own name — and it has one consequence worth remembering: **a tool added to a seed after that agent already exists does not join its grant**. Appendix B lists the tools currently in nobody's toolkit for exactly this reason.

The one-job rule

No agent does more than one task. Read as **one deliverable, one definition of done** — not one tool, and not one department.

The test for anything added to the roster: *does this produce more than one kind of finished thing?* A cold email and a LinkedIn message are one thing in two wrappers. A proposal and an invoice are two things.

It bites harder for agents than for people. An agent holding three jobs has one prompt that must describe all three, one toolkit that is the union of all three, and one memory in which what it concluded about chasing an invoice is recalled while it is writing a proposal. Three separate ways of being worse at each. It also makes the roster answerable: *who chases late payments* is now a card you can pause or re-prompt without touching how proposals get written.

The board and the executive tier are deliberately not split. Their output is a decision or a brief — one deliverable — even where the scope lists four areas. Splitting those would produce a dozen cards that each recommend a fraction of a judgement.

TIER	AGENTS	WHAT THE TIER IS FOR	WHO IS IN IT
Board	1	Sets direction. Produces a decision, never a deliverable.	Board Chair
Executive	7	Owns a function end to end and answers for its numbers.	Chief Executive · Operations Director · Finance Controller · Sales Director · Growth & Content Director ...
Governance	2	Governs how the work is done rather than doing it.	Risk & QA Director · AI People Operations
Operational	9	Runs one stage of the business, day to day.	Agent Creator · Lead Lifecycle Manager · Commercial Operations Manager · Delivery Director · Recurring Revenue Manager ...
Specialist	32	One craft, one deliverable. The ones who make the thing.	Web Developer · Automation Engineer · QA Tester · Graphic Designer · Video Editor ...

BY DEPARTMENT

DEPARTMENT	AGENTS	THE ROSTER
Executive	2	Board Chair · Chief Executive
Delivery	4	Operations Director · Delivery Director · Support Desk · Handover Lead
Finance	6	Finance Controller · Recurring Revenue Manager · Invoice Raiser · Payment Chaser · Renewals Specialist · Forecast Analyst
Revenue	11	Sales Director · Lead Lifecycle Manager · Commercial Operations Manager · Outbound Communications Manager · Proposal Writer · Cold Lead Writer · Lead Capture Runner · Lead Enricher · Deliverability Warden · Growth Analyst · Follow-up Writer
Marketing	12	Growth & Content Director · Graphic Designer · Video Editor · Ad Designer · Copywriter · SEO Specialist · UI/UX Designer · Social Designer · Case Study Writer · Local Search Specialist · Search Intent Researcher · Page Reviewer
Technology	8	Technical Director · Business Intelligence Agent · Automation & Integration Architect · Web Developer · Automation Engineer · QA Tester · Security Analyst · Hosting Engineer
Client Success	5	Client Success Director · Client Communications Agent · Value Reporter · Retention Analyst · Mail Room
Risk & Quality	1	Risk & QA Director
Agent Ops	2	AI People Operations · Agent Creator

The 10 instruction layers

An agent’s prompt is not one block of text. It is 10 named sections, composed in this order, read fresh from the database on every single task.

LAYER	WHAT IT GOVERNS
1 Role role	Who the agent is, and what Dakyworld is. The only layer that says “you are”.
2 Mission mission	The one thing it exists to produce. Matches the mission on its card.
3 Scope scope	What is inside its craft — and the instruction to hand back anything that is not.
4 Data rules dataRules	What it may treat as fact. Shared wording: observed and inferred stay apart, and nothing is invented.
5 Tools tools	That the grant is the limit. The invoker enforces it whatever this layer says.
6 Policy policy	What it may never do on its own. Usually its escalation policy, stated to the model.
7 Process process	How it works through a task. The longest layer, and the one worth editing first.
8 Escalate when escalateWhen	The conditions under which it must stop and ask rather than proceed.
9 Output output	The shape of what it hands back, so a person can tell whether it is finished.
10 Memory memory	What it may keep between tasks. Shared wording: decisions and reasons, never secrets.

Three places must agree on these names — the API that validates an edit, the runner that composes them, and the screen that draws one box per layer — so there is exactly one list, exported as PROMPT_LAYERS.

An edit lands on the next task, not the next restart

The runner reads the row rather than a cached copy. Change the process layer on the Agents screen at 11:04 and the task that starts at 11:05 runs on the new words. POST /agents/:key/prompt/reset restores a seeded agent’s shipped wording without touching its toolkit, autonomy or dry-run setting.

The trap: an instruction that changes nothing

A prompt layer only governs the work if the work actually reads it. Several deliverables used to be written by string constants no screen displayed and no edit could reach — the founder edited a cold-email prompt and the next cold email came out identical. That is what the writer registry in Part 05 exists to close: **every job a model writes now names the agent whose instruction writes it**. Before editing a prompt to change an output, check the writer table in 05.10 that the output is actually owned by that agent.

What an agent is actually told

The assembled prompt has eight labelled regions. Only one of them is authored by a person; the rest are built from live state at the moment the task starts.

1	Its instructions The 10 layers run together in order — or the founder’s own replacement text, if the prompt has been rewritten. The only editable region.
2	What it is relied on for Its skills, as a list. Edited on the Agents screen.
3	Who Dakyworld is The brand block from services/dakyworld.ts. The same for every agent.
4	The company’s details From Settings → System — one record. Change it there and every agent and every document follows.
5	How it writes The voice block. Also the same for every agent.
6	What Dakyworld holds Shared memory recalled for this task’s subjects. Presented as standing instruction : where one conflicts with what the agent would otherwise do, it follows it and says so.
7	What it already knows Its own memory for the same subjects. Presented as its own conclusions : if the record in front of it contradicts one, the record wins and it must say so.
8	How it works here Tool etiquette, what a preview means, what to do when refused, and the escalation order. Added only when the agent is holding a task — a colleague being <i>asked a question</i> gets regions 1–7 and not this one, because telling somebody with no tools how to use them is noise.

Regions 6 and 7 are two headings on purpose. They carry different authority, and an agent that argues with a house rule because it was pasted in under the heading “your own conclusions” is a failure of that paragraph rather than a failure of the agent.

THE ROSTER IS A NUMBER, NOT A LIST

Region 8 tells an agent it has 51 colleagues without naming any of them. Printing 51 names into every prompt would cost more tokens than most tasks; telling it the number is what makes findAgent the obvious next move instead of a tool it never thinks to reach for.

The escalation order, in the words the agent is given
When work needs a craft that is not its own, the answer is never to attempt it anyway. It works down this list and stops at the first step that answers: **1. findAgent** — find whose craft this is, *before* attempting it. **2. consult** — keep the work, ask one question inside their craft. **3. handOff** or *delegate* — the work itself is theirs. **4. needSkill** — only when findAgent found nobody. “Asking is cheap and being wrong in public is not.”

Memory and context are not the same thing

One is what an agent **concluded**. The other is what **happened**. Keeping them apart is what lets an agent be told “your own conclusions lose to the record in front of you” and have that mean something.

Memory — AgentMemory

A decision and why, an outcome, something learnt. Written with `remember`. Two scopes: **AGENT** is its own and outlives only itself; **SHARED** has a null agent key, keeps its author, and is recalled by every agent.

Recall is by subject — `lead:x, client:y, self` — never by similarity. A similarity search would eventually put one client’s facts in another client’s letter. Sharing widens *who sees* a memory, never *when it comes up*.

`findSecret()` refuses credential-shaped content outright, because a memory is re-read into a prompt every time its subject comes up.

Context — the dossier

Everything known about one company, as one document: the audit from `WebsiteAudit`, the letters from `EmailMessage`, the calls from `Communication`, and the notes with no home of their own from `ContextNote`. Written with `addToHistory`, read with `readHistory`.

Assembled at read time, never stored. A fact with two homes eventually has two values — and a log only knows what happened after it was switched on, so every company audited before this shipped would otherwise have an empty dossier, which is exactly when an agent most needs one.

The instruction the agent is given says it plainly: *writing an opinion into the history dresses it up as a fact for somebody who was not there; writing an event into your memory keeps it from the colleague who needs it.*

PART 02

What an agent may do

69 tools in 9 groups, one way of calling any of them, and four checks every call passes before the handler ever runs. Adding a tool means one entry in a catalogue; it inherits the whole policy by existing.

IN THIS PART

The catalogue · The gate, in order · The autonomy ladder · What a preview is · The rationale a spend must carry · The nine built-in tools every agent has · The audit row

The catalogue

A tool is a catalogue entry: a group, a purpose, what integration it needs, a Zod schema, a handler, and a preview. Two flags decide what it costs to be wrong.

69

TOOLS IN CODE

Across 9 groups.

12

OUTWARD

Visible outside the company.
Needs level 3.

16

SPENDING

Costs real money. Needs level 4.

+MCP

CONNECTED SERVERS

Add more tools, under the same gates.

GROUP	TOOLS	OUTWARD	SPENDS	KEYS
Pipeline	8	0	4	lead.read lead.update lead.prepare lead.prepareMany capture.plan capture.cost capture.run capture.spend
Clients	2	0	0	client.read crm.read
Delivery	3	0	0	projects.read tasks.write time.read
Money	6	2	0	finance.read careplan.read analytics.read payment.link payment.momo payment.status
Communication	15	4	3	email.draft email.send sequence.enrol suppression.check sequence.stop inbox.read inbox.route inbox.handled slack.send message.reach message.draft whatsapp.send whatsapp.link whatsapp.templates sms.send
Research	7	1	2	company.audit site.look audit.website audit.read github.read github.issue security.scan
Operations	14	4	0	calendar.read calendar.write webhooks.read repo.read repo.create code.propose code.merge webhook.dispatch agents.read integrations.read agent.gaps agent.roster agent.hire agent.closeGap
Documents	7	0	5	proposal.draft invoice.draft content.draft content.factcheck content.humanise document.render email.polish
Studio	7	1	2	design.brief video.plan ad.concept web.page image.generate demo.build demo.read
Total	69	12	16	

outward and spends are the whole risk model

Everything else in the catalogue is description. Those two booleans decide which autonomy level a call needs, what a rehearsal holds back, what a budget ceiling downgrades, and which calls have to carry a written rationale. A tool author sets them once and never thinks about policy again.

02.2

The gate

`invokeTool` in `services/tools/invoke.ts` is the **only** way a tool gets called. Everything an agent is not allowed to do is enforced here rather than in the tools, in this order.

1 Does the tool exist, and is the integration behind it configured

Readiness. A Slack tool with no Slack connected is refused with a sentence that names the setting, not with “something went wrong”.

2 Is the agent alive, and is it granted this tool

A RETIRED or PAUSED agent is refused outright. The grant is `toolkit`; an empty toolkit can call nothing.

3 Is it over a hard spending ceiling

Only checked for a tool that spends — a blanket hold would stop an agent *reading* a record, which blinds it without saving a penny. **This is the one check an approval cannot lift:** approving a letter is a decision about the letter, not a decision to go over budget. The refusal names the ceiling so it can be raised and the card approved again.

4 Does the autonomy level allow acting, or only preparing

Below level 3 an outward call is downgraded to a preview. Below level 4 a spending call is. This produces `dryRun`, not an error — because a prepared action a person can approve is the useful outcome.

5 Is the input the shape the tool declared

The Zod schema. A model will confidently pass a string where a date was wanted; this catches it before the handler does something odd with it.

6 Has this exact call already been made

Idempotency. For outward tools the runner passes `${taskId}:${tool}:${sha256(input)}`; a successful non-preview call already on record returns its recorded output instead of running again. Opt-in, because two identical sends can be two correct sends — a monthly reminder is the same payload every month. What the runner knows is that a **resumed** run replaying a turn is always a retry.

Every call is written to `ToolCall` — including the refusals. That is the record which answers “why did nothing happen last night”.

What an approval lifts, and what it does not

`approvedRequestId` lifts **only** the prepare-rather-than-act downgrade. Readiness, the grant and the hard ceiling are all still checked, so a request approved after the tool was revoked, or after the integration was disconnected, is refused rather than honoured. That is what separates it from `asOwner`, which skips the checks entirely because a person is driving the tool directly.

The autonomy ladder

Six rungs. Everything on the roster seeds at rung 1 with dry run on, and only a person moves it.

0	Observe	Reads and reports. Calls nothing that writes.
1	Draft	Where everything ships. Researches, reasons and prepares. Outward and spending calls come back as previews, and the task finishes at <code>NEEDS_APPROVAL</code> with the whole thing ready for one decision.
2	Recommend	Writes to our own records and argues for a course of action. Still cannot reach outside the company.
3	Execute with policy	The outward threshold. Sends the email, enrolls the sequence, opens the issue, books the meeting, publishes the page. Its policy layer still binds it.
4	Autonomous	The spending threshold. Starts a scrape, raises a payment link, buys a model call it chose to make. Money is the harder mistake to undo, so it is one rung above acting.
5	Delegated	Reserved. Nothing on the roster is set here.

dryRun sits *above* the level rather than inside it: an agent at rung 4 with dry run on still prepares everything. It is the switch that lets a level be raised for a week and put back without re-editing a card.

A refusal is information, not a failure

When the gate downgrades a call to a preview, the model is told exactly that — in words — and the useful thing then happens: it carries on and prepares the rest of the work instead of stopping. That is why a level-1 agent produces one complete thing to approve rather than five half-finished ones.

THE RATIONALE A RISKY CALL HAS TO CARRY

Outward and spending tools take a rationale of three parts — **why, gain, risk** — and it is required by the schema the model is handed, not by a prompt line that can be forgotten. It is what turns a preview into something a person can actually decide on. *“Send this email”* is not a question anybody can answer; *“send this email, because their certificate expired, which gets us the strongest opening we have, and the risk is that their host caused it”* is.

The nine tools every agent has

These are not in the catalogue and not in any toolkit. They are how an agent relates to the rest of the workforce, and every agent gets them.

escalate

Stop and ask. Ends the task at **BLOCKED** with the question on it. Answering appends to the brief *and* to the conversation, and the run carries on from where it asked.

remember

Keep a decision, an outcome or something learnt, against a subject. Refuses anything credential-shaped.

addToHistory

Write to the company's own account of a client — a call and what was agreed, a decision and what came of it. Every agent that opens the dossier next reads it as evidence.

readHistory

The rest of the dossier beyond the headlines the brief carries: the wording of what was sent, what they said back, anything older.

findAgent

Search the roster by craft. The first move whenever work is outside its own job.

consult

Keep the work, ask a colleague one question inside their craft. They answer from their own instructions and their own memory of this client. **Their answer is an opinion, not a checked fact** — where it contradicts the record, the record wins and the agent must say so.

delegate

Hand work *down* to a direct report. Constrained by `managerKey`.

handOff

Hand the remaining work *sideways* to whoever's craft it is. The brief is written as if to somebody who was not here, because they were not.

needSkill

Record that Dakyworld has no such craft — only after `findAgent` found nobody. Feeds the hiring loop in 05.8. **Not** a way to put down work that is actually its own; a gap raised for something a colleague already does argues for hiring a duplicate.

PART 03

One task, end to end

From the moment a task is raised to the moment a person reads what came of it — the claim, the prompt, the loop, the checkpoint after every single tool call, and the five ways it can end.

IN THIS PART

Where a task comes from · The claim, and why one agent holds one task · The turn-by-turn loop · The timeline · Checkpoints that survive a deploy · The end states · Interrupt, drain, resume, reap

Where a task comes from

Four origins, and they are recorded because “who asked for this” is the first question about anything that went wrong.

ORIGIN	RAISED BY	EXAMPLE
OWNER a person	Somebody opening the Agents screen and giving an agent a brief.	“Audit this site and tell me whether it is worth approaching them.”
SCHEDULE standing work	An AgentSchedule coming round on its own times, in its own timezone.	A daily pipeline brief at 07:00 Accra.
EVENT something happened	A webhook, a status change, a due date, a message arriving in the mailbox.	A prospect replies; the mail room routes it and raises the task.
AGENT a colleague	deLegate from a manager, or handOff to another craft.	The proposal writer hands the pricing question to Commercial Ops.

Raising work and doing work are deliberately separable

Standing work raises tasks on one tick; the runner works them on the *next* one. Keeping those apart means you can watch what the workforce decided to start without it having already started.

The claim

Two ceilings, and the second one is about memory rather than tidiness.

PROCESS CEILING	2 tasks at once across the whole service. One service, one loop, one ceiling.
AGENT CEILING	One agent, one task. Enforced <i>inside</i> the claim as a conditional update — agent: { tasks: { none: { status: "RUNNING" } } } — so two processes reaching for the same agent cannot both win.
REHEARSAL CEILING	1 , below the process ceiling on purpose. A rehearsal is discretionary work and the rest of the business is not.
RUN OWNER	A token identifying this process, stamped on the claim and matched inside every checkpoint write. A process reaped as dead that later wakes finds its token replaced and stops.

The reason for the agent ceiling is worth stating plainly. An agent writes what it concluded as it goes and reads it back on the next task about the same subject. Two tasks about the same lead running side by side interleave those writes, so each recalls half of what the other was in the middle of deciding — and the result is an agent contradicting itself inside one conversation, with nothing in the timeline to show why.

It has one consequence that had to be handled: **a stranded RUNNING row now blocks its agent entirely**. That is what the reaper in 03.6 is for.

The loop, turn by turn

A manual tool-use loop, not the SDK’s tool runner — because every call must route through the gate, each call must write its step immediately, and the loop must stop on the agent’s own `escalate`.

1	Claim Conditional update to <code>RUNNING</code> , stamped with this process’s token. A <code>STARTED</code> step is written.
2	Gather Read the agent row fresh. Work out the task’s subjects. Recall memory for those subjects — shared and own, separately. Assemble the dossier headlines into the brief.
3	Compose The eight prompt regions of <code>O1.4</code> , joined. The tool schemas handed to the model are the agent’s granted toolkit plus the nine built-ins — so a tool it does not hold is not merely refused, it is never offered.
4	Turn One model call. Anything the agent chose to say on the way is written as a <code>THOUGHT</code> step. Checkpoint written.
5	Call Each tool the turn asked for, one at a time, through <code>invokeTool</code> . The result is clipped to a size the next turn can carry. Checkpoint written after every single call , so a resume runs only the calls that never happened.
The ordering that must not come back In the tool wrapper, <code>result.dryRun</code> is checked before <code>result.refusedReason</code> . A dry run carries a refusal reason too — the sentence explaining why it was downgraded — so the wrong order files prepared work as refused, leaves the prepared-call count at zero, and finishes the task <code>DONE</code> . The founder then reads “done” about work that never happened.	
6	Check the stop flags Between iterations and between tool calls: has a person asked this task to stop, has the process been told to shut down, has the iteration cap been reached. All three land the run somewhere legible rather than mid-sentence.
7	Repeat, to a ceiling of 16 iterations The cap counts across resumes . The attempt counter resets on any run that <i>progressed</i> , so the cap catches a task dying in the same place rather than one that met three deploys.
8	Finish The agent’s final message — what it did, what it found, what a person should do next, in plain English. That message is what gets read. The end state is decided from what actually happened, not from what the agent says happened.

THE TIMELINE

Every step is a row in `AgentTaskStep`, written as the run happens rather than assembled afterwards, so a task in flight can be watched. The kinds are: `STARTED` `THOUGHT` `TOOL_CALL` `PREPARED` `REFUSED` `DELEGATED` `CONSULTED` and the hand-off kind — the last two distinguished on purpose, because asking a colleague sideways and giving them the work are different events and a timeline that blurs them cannot be audited.

Checkpoints

A run survives the browser, the deploy and the stop button.

The route that starts a task was always fire-and-forget, so a closed tab never stopped anything. What used to be lost was everything else: a deploy landing mid-task discarded sixteen turns of paid work and began again from the brief.

WRITTEN	After every model turn and after every single tool call .
THE HALF-TURN	A turn whose calls are only partly done lives outside the message list as pendingAssistant + pendingResults — an assistant turn with only some of its results after it is not a conversation the API will accept. A resume therefore runs only the calls that never happened. That is the entire point: “again” for email.send is a second letter to the same prospect.
OWNERSHIP	The run-owner token is matched inside every write.
HEARTBEAT	Touched on every checkpoint write. Five minutes of silence is a run that has genuinely stopped rather than one that is thinking — generous against the slowest single step, which is one high-effort model turn or one tool call that fetches a site, photographs it twice and waits on Apify.
KEPT OR CLEARED	DONE and NEEDS_APPROVAL clear it. BLOCKED , FAILED and CANCELLED keep it — so “carry on” means carry on. Housekeeping prunes the ones nobody was ever going to continue.

The end states

Six, and each one means something different. Reading them as a set is how you tell a working system from a busy one.

STATE	WHAT IT MEANS	WHAT TO DO ABOUT IT
DONE	It finished and its work took effect.	Read the final message.
NEEDS_APPROVAL	It finished, and its work was prepared rather than done — the agent was in dry run, or below the autonomy its tools needed. The normal outcome at the shipped settings.	Open the approvals queue. Approving re-invokes the tool with the input the schema already validated, so what happens is what was shown.
BLOCKED	It stopped and asked. The question is on bLockedReason.	Answer it. The answer must go through the escalation path — onto the conversation as well as onto the brief — or the agent resumes at the moment it asked and asks again.
FAILED	Something broke, or it was interrupted repeatedly without ever getting further.	Read the error. A rate limit is retried up to three attempts before this.
CANCELLED	A person stopped it.	Nothing. Its checkpoint is kept.
QUEUED	Waiting for a runner. The normal resting state — and where an interrupted task goes back to, with its place kept.	Nothing, unless it has been there a long time: check whether its agent is holding another task.

A RUNNING task can be stopped, but not instantly

An interrupt is a request read *between* iterations and *between* tool calls, which returns the task to QUEUED with its place kept. It is not a kill: the loop is not interruptible mid-call, and the route says so rather than lying about it. In testing, **three tool calls finish before a poller can even ask** — which is why the interrupt harness uses twenty rather than six, and why the first version of it reported false failures for a mechanism it never actually exercised.

03.6

Interrupt, drain, resume, reap

Four mechanisms, all pointed at the same failure: a task left RUNNING by a process that is gone, holding an agent that then cannot work at all.

1 Drain — on SIGTERM

The platform gives a container a few seconds to die politely, and this spends them well: every run in flight is asked to stop at its next safe point, checkpoint written, task back to QUEUED, agent freed. Up to an eight-second grace.

2 Resume — at boot, before the first tick

Runs the last process died in the middle of are handed back *before* the first tick, because each one is holding an agent that cannot work until it lets go — and each carries a checkpoint, so handing it back costs nothing but the seconds since the restart.

3 Reap — on every tick

A RUNNING row whose heartbeat has been quiet for five minutes, and which no live process in this instance owns, is requeued. Rows this process is actually running are left alone however long they have been going, which is what makes it safe every minute. A run predating the heartbeat is judged on its start time with the old forty-five-minute timeout.

4 Give up — after three attempts without progress

Requeued rather than failed while attempts remain. The counter resets on any run that moved the conversation forward, so this catches a task that keeps dying in the same place — “something is stopping this run rather than it failing” — and not a task that merely met three deploys.

PART 04

The clock

Nine independent jobs on one sixty-second interval, each settled separately so that one failing cannot stop another. This is the part that happens whether or not anybody opens the app — and therefore the part worth being certain about before an autonomy level is raised.

IN THIS PART

What runs every minute · The advance-before-work discipline · The daily housekeeping · The boot order ·
What a long outage does

Every minute

Deliberately one `setInterval` inside the web process rather than a cron container: Dakyworld OS runs as a single service, and one interval has no moving parts to keep in sync with a deploy.

JOB	WHAT IT DOES ON EACH PASS	IF IT FAILS
Lead capture <code>captureTick</code>	Starts whichever scraper sources are due, by their own local times in their own timezones. Backfills any source that has no next time worked out.	Settled alone. Being at the concurrency limit or over budget is the guardrail working, not a fault — it says so and the next slot tries again.
Care plan billing <code>billDuePlans</code>	Each active plan bills on its own day of the month.	Settled alone, so a failing scrape cannot stop an invoice.
Email dispatch <code>dispatchDueEmails</code>	Sends whatever is queued and due, inside the sending window.	Settled alone.
Phone dispatch <code>dispatchDueMessages</code>	WhatsApp and SMS on the same tick.	Settled separately from email for one specific reason: an expired WhatsApp token must not stop an invoice email.
Sequences <code>runDueSequences</code>	Advances every enrolled contact to its next touch, or ends the series.	Settled alone.
The mailbox <code>readMailboxOnce</code>	Whatever has arrived since the last pass, in both the inbox and Sent.	The live IMAP connection normally beats this to it. This is the floor under it — and the only thing that reads Sent, so a reply the Owner typed on his phone is noticed within the minute.
Agent tasks <code>runDueTasks</code>	Starts queued tasks belonging to ACTIVE agents, up to the ceiling. Also reaps abandoned runs.	Unlike the others this one does not wait : a run can take minutes and holding the tick open for it would stop an invoice going out.
Standing work <code>raiseStandingWork</code>	Raises the tasks agents do without being asked, from <code>AgentSchedule</code> .	Raises only. <code>runDueTasks</code> works them on the <i>next</i> tick, deliberately.
Housekeeping <code>housekeepingTick</code>	Once a day, not once a minute. Six sweeps — see 04.3.	Cheap to check, expensive to run, and nothing depends on it being prompt.

Two rules every scheduled thing follows

The slot is spent before the work starts The next due instant is written to the row and advanced before the work begins. A failure therefore cannot be retried in a loop, and a restart cannot fire the same slot twice. Lead capture, care-plan billing and standing work all do this identically.	A missed slot is skipped, not stampeded Anything more than six hours stale was missed during an outage. It is skipped with a line in the log rather than run six times on boot. Without this, coming back from a long outage would be indistinguishable from a runaway.
---	---

Standing work carries a third ceiling of its own: `maxOpenTasks` per schedule, by default one — so a daily brief that is still unfinished tomorrow does not become two. Combined with the per-agent lock, a loop that raises work faster than an agent finishes it is impossible rather than merely unlikely, which matters because the first sign of that failure would have been the bill.

A DRAFT or PAUSED agent’s standing work is skipped **silently**. Pausing an agent is exactly how the founder stops its standing work, and a warning every sixty seconds about a deliberate decision is noise.

Once a day

Six sweeps, each closing a way the system quietly rots.

SWEEP	WHY IT HAS TO EXIST
Scraper run history	Past its retention window. The check is cheap and the delete is not.
Expired sessions	A session is deleted when it is presented — but a session nobody ever returns to is never presented, so without this the table keeps every token hash the system has ever issued.
Stale checkpoints	Conversations kept so a blocked or failed task could be continued, long after anybody was going to continue one.
Unanswered hire requests	Pending proposals are <i>counted</i> against a ceiling, so five forgotten ones would stop the Agent Creator proposing anything at all with nothing on screen to say why.
Orphaned skill-gap reviews	A gap whose review task somebody cancelled would otherwise sit in review with nothing reviewing it.
Stale approval requests	A week-old proposal is a re-ask, not a refusal — the invoice may have been paid and the lead gone cold since. Carrying one out on the strength of a decision nobody actually made is the mistake this guards against, and leaving them pending would turn the queue into a list of things that are not going to happen, which is how a queue stops being read.

The boot order

1	Migrations, then listen prisma migrate deploy runs as part of the start script.
2	Seed anything new ensureAgents() — creating only, never overwriting.
3	Resume interrupted scraper runs Apify runs that outlived the process that started them.
4	Resume interrupted agent tasks Before the first tick , because each is holding an agent that cannot work until it lets go.
5	Restore orphaned rehearsal wakes Agents a rehearsal woke and never got to put back. Left alone, the next tick hands those agents <i>real</i> work — a test changing how the business runs, days later, with nothing on screen connecting the two.
6	The mailbox watcher connects A long-lived IMAP IDLE connection, renewed every nine minutes, reconnecting with backoff from one second to one minute.
7	First tick, then every sixty seconds The interval is unref'd so it never holds the process open on shutdown.

PART 05

Every process

Nine business stages and eight machine processes. Each one described the same way: what starts it, who owns it, what it calls, what gate it passes, what ends it, and what makes it stop and ask.

IN THIS PART

The nine stages · Capture · The mail room · Outreach and the phone channels · Approvals · Hiring · Rehearsals · Budgets · Model routing and failover · Who writes what

The nine stages, in one table

A business nobody has heard of, to a retainer renewing. Read the right-hand column first: it is the part that happens without anybody pressing anything.

STAGE	OWNER	IN → OUT	RUNS WITHOUT ASKING
1 Acquisition	Lead Lifecycle Integration Architect	A business nobody has heard of → a lead at NEW, scored and de-duplicated	Scheduled capture; webhook intake
2 Qualification	Lead Lifecycle Sales Director	Lead at NEW → QUALIFIED or DISQUALIFIED, with the reason	An audit runs on every new lead
3 Outreach	Outbound Communications	Lead at QUALIFIED → a reply, or the series ends	Enrol, send in window, stop on reply
4 Consultation & proposal	Sales Director Commercial Ops	A reply asking to talk → proposal WON or LOST	Follow-up series after SENT
5 Onboarding	Commercial Ops Delivery Director	Proposal WON → project at PLANNING, tasks assigned	Welcome email on project open
6 Delivery	The specialists	Tasks at TODO → every task DONE, QA signed off	Routing by craft
7 Handover	Client Communications	QA sign-off → project DELIVERED	Testimonial and care-plan series
8 Invoicing	Finance Controller	Milestone or delivery → invoice PAID	Overdue reminders escalate
9 The retainer	Recurring Revenue Client Success	Care plan ACTIVE → renews, or churn is caught early	Billing at 06:00 on the billing day

Capture — how a lead appears

Leads fill themselves. Owner-gated throughout, because runs cost money.

1	A source is configured One Apify actor, its input, its daily times and its timezone. The Apify token lives encrypted in the database rather than in an environment variable — the whole point is adding a source without a redeploy.
2	The tick finds it due Slot advanced first. Skipped if more than six hours stale.

3

The cost ceiling is checked

Every shipped template actor is pay-per-event, so an item cap does nothing — a maximum total charge is the cap that works. **The budget check fails open** if Apify's usage endpoint is unreachable: a monitoring blip must not stop lead generation.

4

The run starts

Proxy configuration is injected only into a field the actor's published input schema actually declares — the Google Maps actors declare none, one contact scraper requires `proxyConfig`, other crawlers use `proxyConfiguration`, and an undeclared key is a hard validation failure. Never guessed.

5

Results are mapped

Candidate-path lists per lead field rather than per-actor code. A capture batch becomes a lead group; each lead gets a dedupe key.

6

Preparation

Research, a website audit, a homepage screenshot, and a polish pass — with guards that stop it inventing a business it could not actually find.

The other way in is a webhook. `/api/webhooks/website-form` is deliberately allowed unsigned, because a static site has nowhere to keep a secret; every other sender must sign with HMAC-SHA256 over `timestamp.body`. The signing secret mints itself, so webhooks need nothing pasted.

05.3

The mail room — what happens to a reply

Somebody answering a cold email is the most time-sensitive event this company produces. The difference between forty seconds and four minutes is the difference between a business that is paying attention and one that is not.

Watch mailbox/watcher.ts	A long-lived connection sitting in IMAP IDLE. It does not read anything — it listens, and asks the sync to do the reading on its own short-lived connection. A socket that is fetching is not idling, and juggling both on one connection is where every home-made IMAP client goes wrong. IDLE is renewed every nine minutes because what kills it is usually a NAT table rather than the server.
Sync mailbox/sync.ts	Reads the new UID range. Guarded by a promise rather than a lock, because a push and a tick arriving together is normal and two passes would double the work and race on the cursor. The minute tick is still the floor — so the worst case of the whole watcher failing is the behaviour we would have had without it.
Parse mailbox/parse.ts	Headers, body, thread. Decides whether a machine wrote it.
Consequences mailbox/consequences.ts	What needs no judgement, in a fixed order: suppression first, then stopping sequences — because stopping a sequence pulls its queued mail and suppression pulls everything else queued for that address too. This must happen even when triage is off, even with no model key, and even when every agent is paused. A sequence that keeps writing to somebody who has already answered is the exact failure this module exists to end, and it must not be contingent on an API key.
Triage mailbox/triage.ts	The only model call in the mail room, and deliberately the smallest decision in it. Four questions: what kind of letter is this (one of sixteen intents), what does it say in a sentence, how quickly does it need an answer, and how sure are you. It does not decide who gets the work and does not write a word that reaches a customer.
Route mailbox/router.ts	A table , not a judgement. Each intent has a destination for a known client and a destination for a stranger, because the same question from a stranger and from a client of two years is one question and two different jobs — one is a sale, the other is support.
Raise agents/runner.ts	An EVENT task for the destination agent, carrying the reason in the words the Inbox screen shows.

Why the classification and the routing are separate

A model that chooses the agent is a model that can hand a complaint from a paying client to the cold-outreach writer, and no amount of prompt wording makes that reliably impossible. A model that chooses between sixteen named intents can be wrong in sixteen ways, all of which are visible on screen, and every one of which maps to a routing decision a person approved once, in advance.

The confidence floor does real work

Below **0.6** the message is labelled and given to nobody, which is the honest outcome for mail the system does not understand. An unrouted message sits on the Inbox screen for a person; a confidently misrouted one starts an agent writing to somebody about the wrong thing.

An out-of-office is not a reply

It carries Auto-Submitted, it arrives within seconds of a send, and treating it as an answer stops the sequence, marks the lead engaged, and loses the prospect in silence. Nothing acts on a message a machine wrote — except a bounce, which is read off the headers rather than off a model.

Three more rules hold the router. **A retired or paused agent is not a destination** — the fallback chain ends at the Mail Room itself, then at nobody, which puts the message on screen for a person, because silently handing work to an agent

nobody has switched on is the same as losing it. **Some letters route to nobody on purpose:** a bounce, an out-of-office and a clear no have already had everything done about them that should be done, and raising a task for each would spend a model call to conclude there is nothing to do. And the **Sent folder is read too**, so a reply the founder typed on his phone stops the machine chasing behind him.

05.4

Outreach, and the leads with a number and no email

Outreach runs as sequences: enrol, send inside the window, stop on reply. Suppression is checked before anything goes out. The doctrine those letters are written to is the cold-email skill rather than the older playbook, and it lives in the owning agent’s process layer — which is the surface an edit actually reaches.

For a lead with a phone number and no address there are two more channels on the same dispatch tick. Both are shaped by one constraint: **the 24-hour window**. Outside it, WhatsApp will only carry an approved template, which is why templates are managed as first-class objects rather than composed per send. SMS has no such window and no such richness, and is treated as the fallback it is.

The demo page offer

Cold outreach can offer a free page built for that specific business. A public page carrying somebody else’s name is an outward act with a reputational edge to it, so it is guarded: the design direction is sourced rather than invented, and publishing is an outward call like any other — level 3, or a preview and a card.

05.5

Approvals — the other half of a preview

The gate has always downgraded a risky call to a preview. This is what makes approving it mean something.

Before this existed, approving a task marked it DONE and the previewed letter was never sent — so “approve” meant “I have read this” and the work still had to be done again by hand. Now an approval **re-invokes the tool with the input the schema validated at proposal time**, so what happens is what was shown, not a second model call that might produce something else.

1	The agent proposes An outward or spending call comes back as a preview. An <code>ActionRequest</code> is filed with the exact validated input and the three-part rationale.
2	It appears where a decision can be made The approvals queue in the app, and as a card in Slack.
3	A person decides Approve or reject, with an optional note. Recorded against a signed-in user or a Slack user id.

4 **It is carried out through the same gate**

Readiness and the grant are re-checked; only the dry-run downgrade is lifted. An approval sitting in the queue after the tool was revoked is refused rather than honoured.

5 **Or it expires**

A week-old proposal is a re-ask, not a refusal.

Idempotency here is not a nicety

Slack retries anything it does not hear back from inside three seconds. A retried Approve that sent a second letter would be the worst possible bug in this file — the prospect gets the same email twice and the record shows one send. So the status transition is a conditional update, exactly as the task claim is.

05.6

Hiring — how the workforce grows itself

Three steps, and keeping them separate is the whole design.

1 **A gap is recorded**

An agent hits work outside its own craft, runs `findAgent`, finds nobody, and calls `needSkill`. That is a demand signal and nothing more.

2 **A hire is proposed**

The Agent Creator reads the gap, checks it is not already somebody's job, designs the agent — all 10 layers, with the **smallest** toolkit that does the job — and files a request.

3 **A hire is decided**

By a person clicking Approve in Slack, or by the standing policy when it is set to automatic. **Only this step writes to the agent table.**

No model ever creates an agent

The Agent Creator's tool files a *request*. One function creates an agent, and it is reached only from a signed Slack interaction, an authenticated API call, or the automatic policy. That separation is not ceremony: **an agent that could create agents directly could grant itself any tool in the catalogue by hiring a copy of itself with a wider one**, and there is no prompt wording that reliably prevents that.

Every hire lands the same way — specialist tier, autonomy 1, dry run on — however it was approved. Automatic decides *who exists*. It never decides what they may do.

CEILING ON CUSTOM
AGENTS

25. Beyond it the Creator is refused and told to escalate.

CEILING PER ROLLING
DAY

3, so a runaway loop stops at three rather than thirty.

PROPOSALS WAITING
AT ONCE

5. It must stop proposing until some are decided — which is why unanswered ones have to expire.

A GAP ALREADY
COVERED

Worse than useless: it argues for hiring a duplicate.

Rehearsals — watching the whole workforce work

Point the workforce at one real website and watch what it does. It wakes the agents it needs and puts them back afterwards.

A rehearsal is three facts — a website, a scenario, and a scratch lead — and everything after that is the ordinary runtime doing what it always does. The root task carries a rehearsal marker and the runner passes it down through every hand-off and delegation, so nothing anywhere in the tree can reach outside the building. It is nudged along faster than the minute tick, because a chain of six agents at one a minute is unbearable to watch. And it can be thrown away, because a rehearsal that could not be deleted would make the second one more expensive than the first.

What a rehearsal holds back

Exactly the outward flag, and nothing else. Sending an email, enrolling a sequence, opening an issue, booking a meeting, raising a payment, publishing a page under a stranger's name. Those stop at a preview whatever autonomy their agent is on. That is the whole of the guarantee.

What it deliberately does not

Reads, because a read has no preview and blanket dry-run would have refused every one of them — every agent blind, the timeline full of refusals, and the rehearsal a test of nothing. **Writes to our own records**, which land on the scratch lead. And **spending**: a research pass and a site audit cost real money and there is no honest way to rehearse them without paying. What the run cost is totalled on screen instead — which is worth knowing before the workforce is turned loose on four hundred leads.

Two ceilings: **one task at a time** for the whole rehearsal, below the process ceiling so a wide scenario cannot fill the floor and leave real work queued behind it; and **24 tasks** for the whole run, which is a spending limit rather than a safety one — a scenario that delegates in a circle would otherwise be discovered on the invoice.

The one to keep an eye on is the capture tool: it spends without being outward, so a brief that talked an agent into scraping a city would really scrape it. Nothing in the shipped scenarios asks for that, and the run ceiling plus the cost readout is what would show it.

Budgets — what it may spend before it stops itself

The one part of this system that spends without anybody pressing a button — on a schedule, all night, across 51 agents — used to be the part with nothing above it, and the only symptom of a runaway would have been an invoice at the end of the month.

50% Warn	Say so on screen. Change nothing.
75% Downgrade	Keep working, on the cheap model.
90% Approve	Prepare outward and spending work; do not carry it out. Reuses the dry-run preview rather than inventing a third way for work to be held.
100% Pause	Start nothing new. The one refusal an approval cannot lift.

The two in the middle are the point. Between them the workforce spends the last quarter of its budget doing the same work more cheaply and more carefully, rather than falling off a cliff at the end of the month. A ceiling that can only stop work is a ceiling nobody dares set low enough to be useful.

Several budgets can apply at once — a scope for the agent, for the tool, for the vendor — and the strictest wins. A missing budget is not a zero budget: nothing is enforced until a limit is set, because shipping a default ceiling would stop a working business on the day it deployed.

05.9

Model routing, and what happens when a vendor is down

The caller names the job, never the vendor. Moving every piece of writing in the system from one provider to another is a settings change rather than a diff.

JOB	DEFAULT → FALLBACK	WHAT IT COVERS
Writing text	openrouter → anthropic	Every piece of prose the system produces — proposal copy, email drafts, ad concepts, page copy, cold outreach.
Reading sheets spreadsheet	openrouter → anthropic	Reads an imported spreadsheet of leads — where every table starts and stops, what each column means, which columns don't fit — and returns the plan a person reviews before anything is written.
Sorting a prompt organise	openrouter → anthropic	Reading a written instruction and filing it under the ten headings an agent prompt is made of — so a pasted playbook becomes a prompt rather than a wall of text.
Reading the post triage	openrouter → gemini	Reading an email that arrived and saying what it is — a yes, a question, a complaint, an out-of-office — so it can be handed to the agent whose job it is. See services/mailbox/.
Images image	openai → openai	Pictures for ads, social posts and mock-ups.
Web pages html	openrouter → anthropic	Complete HTML pages on the brand design system — the thing a developer opens and edits.
Fact-checking factcheck	openrouter → anthropic	Checks a draft's claims against live sources, so nothing goes out that stopped being true last year.

Research research	openrouter → anthropic	Finds out who a prospect actually is — trade, address, reputation, who runs it — from live sources, and fills the blanks a scrape left behind.
Plain English humanise	openrouter → anthropic	Rewrites a draft to sound like a person wrote it and to be understood on one reading.
Looking vision	openrouter → anthropic	Reads a screenshot of a prospect's homepage and says what a first-time visitor actually sees — the half of a site audit that markup cannot answer.

Every provider is spoken to over plain HTTP rather than an SDK — three more SDKs would be three more dependency trees and three more ways for a transitive package to break a deploy, in exchange for wrapping one POST each. The exception is Anthropic, which keeps its SDK because the agent loop needs streaming, tool use and thinking blocks.

Structured output is a contract, not a hope. All the vendors support a JSON schema on the response in three different shapes; each adapter translates the one schema the caller wrote into the shape its vendor wants, and a caller never sees the difference.

Failover, and the one failure never routed around

When the assigned model fails, the next capable one in the chain takes over. The exception is a refusal: a model declining to do something is an answer, and routing around it until somebody agrees is how a policy gets laundered. The trap that made this look healthy when it was not: a stubbed vendor that returns success makes a broken provider read green — so the harness has to assert on what came back, not on whether something came back.

05.10

Who writes what

16 writing jobs, each with exactly one owner. **An agent may own several jobs; a job has exactly one owner.**

This exists to close a specific defect. The Agents screen showed a prompt, the founder edited it, and the next cold email came out identical — because the email was never written by that prompt. It was written by a string constant no screen displayed and no edit could reach. The roster was a set of labels over a set of hard-coded writers. The doctrine was in two places at once, and the editable one was not the one that ran.

WHAT GETS WRITTEN	WHOSE INSTRUCTION WRITES IT	WHAT EDITING THAT INSTRUCTION CHANGES
Cold email email.cold	Cold Lead Writer outreach.writer	The first email to a business that has never heard of Dakyworld.
Follow-up email email.followup	Follow-up Writer outreach.followup	The second and later emails to somebody who did not reply.
Invoice email email.billing	Payment Chaser billing.collector	The letters that deliver an invoice and chase a late one.
Client email email.client	Client Communications Agent client.notifier	Project updates, handovers, onboarding and thank-yous — everything written to a client we already have.
WhatsApp and SMS message.phone	Cold Lead Writer outreach.writer	The short messages sent to a phone rather than an inbox.

Proposal proposal	Proposal Writer proposal.writer	The argued document that asks a business for money.
Audit — what a visitor sees audit.ux	Page Reviewer review.look	The reviewer that reads the screenshots and says what stops a first-time visitor.
Audit — speed and findability audit.speed	SEO Specialist seo.specialist	How the measured speed and SEO numbers are explained. The findings themselves are measured, never written.
Audit — the words audit.content	Copywriter content.writer	The reviewer that judges whether the page's words do the selling.
Audit — the verdict audit.synthesis	Cold Lead Writer outreach.writer	The single verdict over the four reviews, and the sentence the email is written from.
Dakyworld's own copy content.draft	Copywriter content.writer	Posts, landing pages and one-pagers written for Dakyworld itself, through the content.draft tool.
The plain-English rewrite content.plain	Copywriter content.writer	The content.plain tool, which rewrites business writing so it reads on one pass.
Demo page demo.page	Web Developer dev.web	The free landing page built for one prospect, carrying their name in public.
Lead research lead.research	Lead Enricher lead.enricher	What is found out about a business before anybody writes to it.
Reading the post mail.triage	Mail Room mail.room	How each message that arrives is read: what kind of letter it is, how urgent, and whether anybody has to answer.
Homepage read homepage.look	Page Reviewer review.look	The first read of a homepage, before the full audit.

Two owners for one deliverable would be the contradiction that makes a model fall back to the generic output it already knew, which is why the rule runs one direction only.

PART 06

All of it, switched on

One ordinary day with every process running: what fires, in what order, what stops for a person, and what is still waiting to be connected.

IN THIS PART

A day, hour by hour · What is on by default · What has to be raised deliberately · What is still to be pasted

A day

Times are illustrative; everything else is the real sequence.

06:00	Care plans bill Each active plan on its own day of the month. Invoices are drafted; sending is an outward act, so at the shipped settings the letters wait in the queue.
06:30	A capture source comes due Slot advanced first. Cost ceiling checked. The Apify run starts; results map into leads with dedupe keys, in one lead group.
07:00	Standing work raises the day's briefs Only for ACTIVE agents, only where the schedule has fewer than its open-task ceiling. Nothing is <i>worked</i> yet.
07:01	The runner picks up the first two Two at a time across the service, one per agent. Each claims, composes its prompt from live state, and starts turning.
07:14	An agent needs a craft that is not its own <code>findAgent</code> , then <code>consult</code> for one question, or <code>handOff</code> for the whole thing. If nobody has the craft, <code>needSkill</code> files a gap.
08:00	The sending window opens Queued email dispatches; sequences advance; WhatsApp and SMS go on the same tick, settled separately.
08:41	A prospect replies The IMAP watcher pushes within seconds. Consequences first — suppression, then the sequence stops — and those happen even if triage is off and every agent is paused. Then triage classifies it, the routing table picks the destination, and an EVENT task is raised.
08:42	The follow-up writer picks it up Its prompt carries the dossier headlines: the audit from June, the letter that was sent, what they said back. It reads the rest with <code>readHistory</code> before writing a word.
08:51	It prepares a reply and stops The send is outward and the agent is at level 1. The call comes back a preview with a rationale attached, the rest of the work is prepared around it, and the task ends <code>NEEDS_APPROVAL</code> . A card appears in Slack.
08:53	The founder approves it The exact validated input is re-invoked through the same gate. Readiness and the grant are re-checked; only the dry-run downgrade is lifted. The letter goes.
11:20	A deploy lands mid-task SIGTERM. Every run in flight is asked to stop at its next safe point: checkpoint written, task QUEUED, agent freed. On boot the resume hands them back <i>before</i> the first tick, and each carries on from the last completed tool call rather than from the brief.

14:00 A rehearsal is started

One real website, one scenario, one scratch lead. Outward calls preview; reads, our own writes and real spending all happen. One task at a time, 24 for the run, cost totalled on screen.

16:30 Spend passes 75% of a ceiling

Work continues on the economy model. At 90% outward and spending calls start preparing instead of acting; at 100% nothing new starts.

21:00 Housekeeping

Run history, expired sessions, stale checkpoints, unanswered hire requests, orphaned gap reviews, and approval requests nobody decided on.

every The reaper

60s

Any RUNNING row quiet for five minutes that no live process owns goes back to QUEUED with its place kept — because one dead row is not one lost job any more, it is an employee who never comes back to their desk.

06.2

What is on, and what a person still decides

Running by itself

Lead capture on schedule. Webhook intake. The mailbox, read live and on the tick, including Sent. Suppression and sequence-stopping. Sequence advance. Care-plan billing. Standing work. Task execution, resume and reaping. Housekeeping. Budget thresholds. Model failover.

Waiting for a person

Every outward act at the shipped settings — each email, each enrolment, each published page, each payment link. Every hire. Any autonomy level above 1. Any budget ceiling, since none is enforced until one is set. And every message triage was not confident about.

Two things that make a refusal test lie

The no-auth development mode means every request arrives as the Owner, and an Owner-driven call skips the grant and autonomy checks entirely. A permissions test that passes there has proved nothing. **And an unset budget is not a zero budget** — a ceiling that was never configured does not refuse anything, so “the budget did not stop it” is usually the honest answer that no budget existed.

STILL TO BE CONNECTED

Everything below is coded, gated and inert until a value is pasted into Settings, where it is encrypted at rest. None of it needs a redeploy. A tool whose integration is unconfigured is refused at the first check with a sentence that names the setting — which is the whole reason readiness is checked before anything else.

The Google connection is the one exception worth flagging: the calendar tools need **no new key**, but an older refresh token does not carry the calendar scopes, so the connection has to be redone rather than topped up.

The roster

All 51 agents as they seed, with the toolkit each one is granted. Read out of `services/agentRegistry.ts` at build time.

AGENT	MISSION	SEEDED TOOLKIT
BOARD		
Board Chair board.chair Executive	Own Dakyworld's long-term direction and protect it from reckless automation.	analytics.readfinance.readcrm.read
EXECUTIVE		
Chief Executive ceo Executive · to Board Chair	Turn board strategy into weekly priorities and keep every department aligned.	analytics.readcrm.readprojects.readfinance.read
Operations Director coo Delivery · to Chief Executive	Keep the agency machine running: processes, handoffs, capacity and SLAs.	projects.readtasks.writetime.readcalendar.read
Finance Controller cfo Finance · to Chief Executive	Report the cash position and name what needs a decision about it.	finance.readcareplan.readanalytics.read
Sales Director cro Revenue · to Chief Executive	Turn qualified opportunities into profitable clients, on evidence rather than volume.	crm.readlead.readproposal.draftcalendar.read
Growth & Content Director cmo Marketing · to Chief Executive	Create demand and strengthen Dakyworld's positioning with defensible claims.	content.draftanalytics.readclient.read
Technical Director cto Technology · to Chief Executive	Own architecture, reliability, security and the evolution of Dakyworld's own systems.	github.readintegrations.readsecurity.scananalytics.read
Client Success Director cco Client Success · to Chief Executive	Keep clients informed, satisfied, retained and moving toward measurable outcomes.	client.readcareplan.readprojects.reademail.draft
GOVERNANCE		
Risk & QA Director risk.qa Risk & Quality · to Board Chair	Stop bad automation becoming bad business.	company.auditsecurity.scanintegrations.read
AI People Operations people.ops Agent Ops · to Board Chair	Manage the agents themselves: performance, permissions, training and retirement.	agents.readanalytics.read
OPERATIONAL		

Agent Creator people.recruiter Agent Ops · to AI People Operations	Decide whether a reported skill gap needs a new agent, and design the one it needs.	agent.gapsagent.rosteragents.readagent.hireagent.closeGap
Lead Lifecycle Manager Lead.orchestrator Revenue · to Sales Director	Score and qualify a prospect against the evidence on the record, and route it to its next step.	lead.readlead.updateaudit.read
Commercial Operations Manager commercial.ops Revenue · to Sales Director	Turn a qualified opportunity into a priced, accurate proposal.	proposal.draftdocument.render
Delivery Director delivery.director Delivery · to Operations Director	Plan accepted work into milestones and assignments, and keep them honest as it runs.	projects.readtasks.write.time.read
Recurring Revenue Manager careplan.manager Finance · to Finance Controller	Bill each retainer correctly: included hours used, overage owed, nothing invented.	careplan.readinvoice.draft.time.read
Outbound Communications Manager email.sequencer Revenue · to Sales Director	Run the outbound sequences: who is enrolled, what goes next, and when a sequence stops.	email.draftsequence.enrolsequence.stop
Client Communications Agent client.notifier Client Success · to Client Success Director	Tell each client what is happening on their project, before they have to ask.	email.draftclient.read.projects.read
Business Intelligence Agent analytics.engine Technology · to Technical Director	Report what the operating numbers actually say happened, with the source behind each one.	analytics.readfinance.read.crm.read
Automation & Integration Architect integration.manager Technology · to Technical Director	Design how Dakyworld's systems connect so information moves automatically and safely.	webhooks.readintegrations.readwebhook.dispatch
SPECIALIST		
Web Developer dev.web Technology · to Technical Director	Build and fix the pages Dakyworld ships.	web.pagedemo.builtdemo.readgithub.readgithub.issuesecurity.scancompany.audit.site.lookaudit.websiteaudit.read.projects.readtasks.write
Automation Engineer dev.automation Technology · to Technical Director	Remove manual admin: map a workflow, wire the systems together, and prove the result is fewer human steps.	webhooks.readwebhook.dispatchintegrations.readgithub.read.projects.readtasks.write

QA Tester qa.tester Technology · to Technical Director	Find what is broken before a client does.	company.audit security.scan github.issue projects.read tasks.write
Graphic Designer design.graphic Marketing · to Growth & Content Director	Make the artwork a client keeps: documents, print and presentation, all on the brand system.	design.brief image.generate document.render content.draft client.read
Video Editor video.editor Marketing · to Growth & Content Director	Turn footage into something worth watching to the end, cut for the platform it will be watched on.	video.plan content.draft client.read
Ad Designer ads.designer Marketing · to Growth & Content Director	Make paid social that earns its click, and test it honestly.	ad.concept image.generate content.draft analytics.read
Copywriter content.writer Marketing · to Growth & Content Director	Write the copy on the page: what it says, in what order, in Dakyworld's voice.	audit.website audit.read content.draft client.read projects.read analytics.read
SEO Specialist seo.specialist Marketing · to Growth & Content Director	Find and fix the technical faults that stop a site being indexed, crawled and ranked.	audit.website audit.read company.audit security.scan site.look content.draft analytics.read lead.read
UI/UX Designer design.ux Marketing · to Growth & Content Director	Design the page a first-time visitor should have seen: what goes where, and why in that order.	audit.read demo.read design.brief lead.read
Security Analyst sec.analyst Technology · to Technical Director	Check what a stranger can see from outside — the certificate, the headers, the cookies, the mail domain — and report only what was actually observed.	audit.website audit.read security.scan company.audit github.issue
Proposal Writer proposal.writer Revenue · to Commercial Operations Manager	Write the proposal that wins the work: what the client actually said they need, what Dakyworld will do about it, what it costs and what happens next.	proposal.draft content.draft content.factcheck content.humanise document.render lead.read lead.prepare client.read projects.read careplan.read company.audit site.look
Cold Lead Writer outreach.writer Revenue · to Outbound Communications Manager	Write the first message to somebody who has never heard of Dakyworld — short, specific to them, and worth the thirty seconds it asks for.	lead.read lead.prepare lead.prepareMany company.audit site.look security.scan content.draft content.factcheck content.humanise email.draft email.polish demo.read audit.read suppression.check analytics.read message.reach message.draft whatsapp.link whatsapp.templates
Support Desk support.desk Delivery · to Operations Director	Answer quickly, fix what is routine, and route the rest to the right person before an SLA is at risk.	client.read projects.read tasks.write email.draft careplan.read

Lead Capture Runner lead.capture Revenue · to Lead Lifecycle Manager	Run the searches that bring new businesses in, at a price that was known before the run started.	capture.plan lead.read	capture.cost	capture.run	capture.spend
Lead Enricher lead.enricher Revenue · to Lead Lifecycle Manager	Fill in what a scrape left blank, from sources that can be cited.	lead.read company.audit	lead.update	lead.prepare site.look	lead.prepareMany
Invoice Raiser billing.invoicer Finance · to Finance Controller	Raise an invoice that matches what was actually delivered.	invoice.draft projects.read	document.render time.read	client.read careplan.read	
Payment Chaser billing.collector Finance · to Finance Controller	Get an overdue invoice paid without costing Dakyworld the client.	finance.read message.reach	client.read message.draft	email.draft sms.send	email.polish
Handover Lead delivery.handover Delivery · to Delivery Director	Hand a finished project over so the client can run it without us.	projects.read content.draft	tasks.write	client.read	document.render
Renewals Specialist careplan.renewals Finance · to Recurring Revenue Manager	Renew a care plan before it lapses, on evidence of what it delivered.	careplan.read	client.read	analytics.read	email.draft
Value Reporter careplan.reporter Client Success · to Recurring Revenue Manager	Write the monthly report that shows a retainer client what they got for the money.	careplan.read analytics.read	client.read document.render	projects.read content.draft	time.read
Deliverability Warden email.deliverability Revenue · to Outbound Communications Manager	Protect Dakyworld's ability to send email at all.	suppression.check company.audit	sequence.stop	analytics.read	
Retention Analyst analytics.churn Client Success · to Business Intelligence Agent	Spot a client who is about to leave, early enough to do something about it.	analytics.read projects.read	client.read crm.read	careplan.read	
Growth Analyst analytics.upsell Revenue · to Business Intelligence Agent	Find the work an existing client already needs, from what the record already shows.	analytics.read projects.read	client.read crm.read	careplan.read	
Forecast Analyst finance.forecast Finance · to Finance Controller	Say what cash and revenue look like in the next three months, and how confident that is.	finance.read	careplan.read	analytics.read	crm.read
Hosting Engineer dev.hosting Technology · to Technical Director	Keep the sites Dakyworld runs online, reachable and recoverable.	company.audit integrations.read	security.scan	github.read projects.read	github.issue tasks.write
Social Designer design.social Marketing · to Growth & Content Director	Make the templates a month of posts can be built from.	design.brief	image.generate	content.draft	client.read
Case Study Writer content.casestudy Marketing · to Growth & Content Director	Turn a finished project into a case study every number of which is true.	projects.read content.factcheck analytics.read	client.read content.humanise	content.draft document.render	

Local Search Specialist seo.local Marketing · to Growth & Content Director	Make a business findable by the people standing near it.	company.audit content.draft audit.read lead.read client.read
Search Intent Researcher seo.keywords Marketing · to Growth & Content Director	Work out what the customers of a business actually type, and brief a page against it.	audit.read content.draft analytics.read client.read
Page Reviewer review.look Marketing · to Growth & Content Director	Look at what a page actually looks like, and say what a first-time visitor takes from it.	site.look audit.read demo.read lead.read client.read
Follow-up Writer outreach.followup Revenue · to Outbound Communications Manager	Write the follow-ups to a message that got no reply, and know which ones not to send.	lead.read audit.read email.draft email.polish content.humanise suppression.check demo.read analytics.read message.reach message.draft whatsapp.link
Mail Room mail.room Client Success · to Client Success Director	Make sure every message that arrives is in front of the person or agent who owns it, the same day it lands.	inbox.read inbox.route inbox.handled lead.read client.read email.draft

read only

 runs at any level

outward

 needs level 3

spends

 needs level 4

The catalogue

All 69 tools, what each needs before it will run, what it costs to be wrong, and how many agents hold it. Read out of `services/tools/catalogue.ts` at build time.

KEY	WHAT IT DOES	NEEDS	GATE & REACH
PIPELINE			
Lead.read	List or look up leads, with their score, status and contact details.	None — our own data	read only held by 12
Lead.update	Change a lead's status, score, notes or estimated value. Never deletes and never rewrites contact details.	None — our own data	preview held by 2
Lead.prepare	Researches a lead from live sources, fills the blanks their scrape left, checks their site and mail domain, and photographs their homepage. What an email has to be written from.	A model key	spends held by 3 preview
Lead.prepareMany	The same as looking at one business, for many at once — and the only efficient way to do it, because the screenshots go into as few Apify runs as possible.	A model key	spends held by 2 preview
capture.plan	Read an instruction like “dental clinics in Kumasi” and say what would be captured. Costs nothing and runs nothing.	Anthropic key	spends held by 1
capture.cost	What a capture would cost at Apify's published rates, before anything runs.	Apify token	read only held by 1
capture.run	Start a configured lead source on Apify. Spends money on every call.	Apify token	spends held by 1 preview
capture.spend	This month's Apify spend against the budget, so an agent can say no before the budget does.	Apify token	read only held by 1
CLIENTS			
client.read	Clients, their contacts, and what is live for each of them.	None — our own data	read only held by 21
crm.read	Counts by stage across leads, proposals and projects — the shape of the pipeline in one call.	None — our own data	read only held by 7
DELIVERY			

projects.read	Projects with their milestones, tasks and assignments.	None — our own data	read only held by 18
tasks.write	Add a task to a project, or move an existing one. Internal only — nobody outside sees it.	None — our own data	preview held by 8
time.read	Hours logged, by project or by person, for capacity and for retainer overage.	None — our own data	read only held by 5
MONEY			
finance.read	Invoices, what is overdue, and what has been paid.	None — our own data	read only held by 6
careplan.read	Retainers: tier, fee, included hours, next billing date and health.	None — our own data	read only held by 11
analytics.read	Revenue, recurring revenue, pipeline value and what the agents have spent, computed from source records.	None — our own data	read only held by 20
payment.link	Opens a Paystack payment page for an invoice and puts the link on the record. The client can pay by card, mobile money or bank transfer. Use this when they are comfortable paying on the web, or when you are putting a link in an email.	Paystack	outward held by 0 preview
payment.momo	Opens a Hubtel checkout for an invoice, so the client can pay from the phone they already have open. Use this where a hosted web page is the wrong ask — which for a great many small businesses here it is.	Hubtel	outward held by 0 preview
payment.status	Asks the provider directly whether an invoice has been paid, and marks it paid if it has. Reading only — it never asks anybody for money.	None — our own data	read only held by 0
COMMUNICATION			
email.draft	Write an email into the outbox as a draft. Nothing leaves until it is sent.	A mailbox	read only held by 9
email.send	Sends a drafted email. This one actually reaches a person.	A mailbox	outward held by 0 preview
sequence.enrol	Puts a lead or client into an email sequence. The sequence sends on its own schedule afterwards.	A mailbox	outward held by 1 preview
suppression.check	Whether an address has unsubscribed, bounced or complained. Always call before writing to anyone.	None — our own data	read only held by 3

sequence.stop	Stops every sequence running at a lead or client — what happens the moment they reply.	None — our own data	read only held by 2
inbox.read	What has arrived: one message in full, a whole conversation, or everything nobody has dealt with yet. Read this before answering anybody — it is the only place a reply they sent actually is.	None — our own data	read only held by 1
inbox.route	Give one message to the agent whose job it is, with a sentence saying why. Search the roster with findAgent first — handing work to the wrong colleague is worse than leaving it for a person.	None — our own data	read only held by 1
inbox.handled	Say a message has been dealt with, and how. A message nobody closes stays on the Inbox screen for ever, which is deliberate — that screen is the list of what is still owed a reply.	None — our own data	read only held by 1
slack.send	Internal alerting and escalation. Goes to the team, not to a client.	Slack	preview held by 0
message.reach	Says which channel a lead can actually be reached on — email, WhatsApp, a text — and why. Reading only. Ask this before writing anything, because it is the difference between an email nobody can receive and a message that lands.	None — our own data	read only held by 3
message.draft	Writes a short message for a phone, from what is on the record. Returns the words and never sends them. Forty words on WhatsApp, fewer by text — a message on somebody's phone is not a shorter email.	A model key	spends held by 3
whatsapp.send	Sends one WhatsApp message. Inside 24 hours of their last reply it carries what you wrote; outside it, only a template Meta approved in advance — name one, or use whatsapp.link instead and let a person send it.	WhatsApp Business	outward spends preview held by 0
whatsapp.link	Writes the message and returns a wa.me link a person opens and sends from their own WhatsApp. No template approval, no fee, and it arrives from a human being rather than a brand — which is what a small business here actually replies to. Nothing is sent by this tool.	None — our own data	read only held by 2

whatsapp.templates	The templates Meta has approved, with how many variables each takes. Outside the 24-hour window these are the only things that can be sent, so read this before writing a first WhatsApp to anybody.	WhatsApp Business	read only	held by 1
sms.send	Sends one SMS. For a payment reminder or an appointment confirmation — the things that get read. Costs money per 160 characters and cannot be recalled, so it is the last resort, not the first: check message.reach before reaching for it.	Hubtel SMS	outward spends preview	held by 1
RESEARCH				
company.audit	Fetches a prospect's site and checks DNS, mail records and TLS. The evidence a proposal argues from.	None — our own data	read only	held by 11
site.look	Photographs a homepage and says what a first-time visitor actually sees — the half of a site review that reading the markup cannot answer.	Apify token	spends preview	held by 6
audit.website	Four reviewers over one site — UI/UX, speed and findability, content, security — compiled into one report, with the findings drawn onto a screenshot of the homepage. Produces a branded PDF and a Markdown copy.	Apify token	spends preview	held by 4
audit.read	What the audit team found on a site, as the Markdown report — the evidence an email or a proposal argues from.	None — our own data	read only	held by 11
github.read	Repositories, recent commits and open issues — deployment and delivery context.	GitHub token	read only	held by 4
github.issue	Raises an issue on a repository. The only thing an agent may write to GitHub.	GitHub token	outward preview	held by 4
security.scan	The security half of a company audit: TLS, mail authentication, DNS and exposed platform details.	None — our own data	read only	held by 8
OPERATIONS				
calendar.read	What is booked, and which parts of a window are free.	Google + calendar scope	read only	held by 2

calendar.write	Creates a calendar event. Invitees are emailed by Google, so this reaches people.	Google + calendar scope	outward held by 0	preview
webhooks.read	What other systems have sent us, and whether it was acted on.	None — our own data	read only held by 2	
repo.read	Lists a folder in a repository, or reads one file. For understanding a codebase before proposing a change to it.	GitHub token	read only held by 0	
repo.create	Opens a new private repository, for a client project that is starting.	GitHub token	outward held by 0	preview
code.propose	Opens a branch, commits a set of files to it, and raises a pull request explaining the change. Changes nothing that is live — merging is a separate decision a person makes.	GitHub token	preview held by 0	
code.merge	Merges a pull request. On this repository that deploys to production, so it always needs a person to approve it.	GitHub token	outward held by 0	preview
webhook.dispatch	Posts a signed event to another system. Leaves the building, so it is dry-run gated.	None — self-configuring	outward held by 2	preview
agents.read	Which agents exist, what they may do, and what their calls have cost.	None — our own data	read only held by 2	
integrations.read	Which integrations are connected and which are waiting on a key — what the agents can actually reach.	None — our own data	read only held by 5	
agent.gaps	Every craft an agent has said Dakyworld does not have, and how many separate agents have asked for it.	None — our own data	read only held by 1	
agent.roster	Who already does a kind of work, matched on what they are good at rather than on a tool key. The check before proposing a hire.	None — our own data	read only held by 1	
agent.hire	Files a complete design for a new agent. It does NOT create one — the Owner approves it in Slack, or the standing hiring policy does. Nothing exists until then.	None — our own data	preview held by 1	
agent.closeGap	Records that a reported gap is not a new agent — the work belongs to somebody who already exists, or Dakyworld does not do it.	None — our own data	preview held by 1	

proposal.draft	Writes a proposal from the audit and the record, into the pipeline as a draft. Nothing is sent.	Anthropic key	spends held by 3
invoice.draft	Creates an invoice in DRAFT against a client. Never sends it and never charges anything.	None — our own data	preview held by 2
content.draft	Writes marketing or client-facing copy in Dakyworld's voice. A draft for a person to approve, never published.	A model key	spends held by 14
content.factcheck	Reads a draft against live sources and says which claims still hold, which have gone stale, and which cannot be evidenced at all.	A model key	spends held by 3
content.humanise	Rewrites a draft so it reads like a person wrote it and lands on one reading. Keeps every fact; changes only how it is said.	A model key	spends held by 4
document.render	Turns a proposal or an invoice into a branded PDF and returns its size and where it went.	None — our own data	read only held by 7
email.polish	The last read before a person sees a draft: makes it sound like somebody wrote it, keeps every fact exactly as it was, and says whether the email actually does its job.	A model key	spends held by 3
STUDIO			
design.brief	Turns a request for artwork into a brief a designer can work from: purpose, audience, sizes, hierarchy, colour and the exact copy.	Anthropic key	read only held by 3
video.plan	Turns raw footage and an objective into an edit plan: structure, shot list, on-screen text, captions, music direction and the cut per platform.	Anthropic key	read only held by 1
ad.concept	Paid-social concepts: the hook, the visual, the primary text, the headline and the call to action, sized per platform.	Anthropic key	read only held by 1
web.page	Produces a complete, self-contained HTML page on the brand design system — the thing a developer opens and edits rather than starts from nothing.	A model key	read only held by 1
image.generate	Makes a picture. Goes to ChatGPT when a key is set, and to a connected MCP image server otherwise.	A model key	spends preview held by 3

demo.build	Builds a landing page for one prospect — their name, trade and services — from what the scan found, against design references from real published work. Hosted at /demos/<slug>.	A model key	outward spends preview
		held by 1	
demo.read	What has been built for whom, whether the link has gone out, and whether they opened it.	None — our own data	read only
		held by 5	

11 tools sit in nobody's seeded toolkit

They exist, they are gated, and no agent can reach one. That is the ordinary consequence of ensureAgents() only ever creating: a tool added after an agent was seeded does not join its grant. It is also the list to work down the day an agent reports that something cannot be done.

email.send · slack.send · calendar.write · payment.link · payment.momo · payment.status · whatsapp.send · repo.read · repo.create · code.propose · code.merge

69 tools in code. 12 reach outside the company; 16 spend money. Connected MCP servers add more, under the same gates.